1. Introduction Algorithmic trading systems have a well-documented failure mode: strategies that perform well in historical backtests often degrade when market regimes change. The traditional response is to retrain models with recent data, but this frequently accelerates the problem by overfitting to short-term patterns [see Section 3.4]. This paper documents Organismo Vivo ("Living Organism"), a multi-agent trading framework developed over several years through close collaboration between a human trader and multiple AI assistants. The project's central thesis is that the price of any asset breathes: it expands and contracts in cycles, and these cycles can be measured objectively. If measurable, they can be traded with rather than against. This intuition led to the formalization of "respiratory states" (hyperactivity, latency, fibrillation, apnea) as objective market regimes—analogous to physiological states in biological systems. The framework evolved from rigid rule-based systems (Snake, 2018-2020) to learned representations (Organismo Vivo, 2024-present), incorporating reinforcement learning, contrastive encoders, and meta-agent supervision. The paper makes three primary contributions: Methodological: We document a multi-agent framework combining reinforcement learning agents with encoder-based regime detection, validated through walk-forward testing, multiple comparison corrections, and preregistration of hypotheses. Empirical: We report results—including edge degradation patterns—with explicit acknowledgment of limitations. We document seven production bugs found through adversarial auditing, including a critical clock-offset bug that explained the divergence between shadow and live performance. Philosophical: We argue that manual filters consistently underperform learned representations in adaptive systems, that retraining with recent data degrades performance (contrary to common practice), and that psychological discipline—not technology—remains the primary risk in systematic trading. The paper is organized as follows. Section 2 presents the project's evolution from rigid rules to adaptive systems. Section 3 describes the methodology, including the respiratory state framework, the encoder architecture, and the validation strategy. Section 4 documents our adversarial audit process and the bugs it uncovered. Section 5 discusses the philosophical decisions that shaped the system. Section 6 reports results with appropriate caveats. Section 7 enumerates limitations and lessons learned. Section 8 outlines future work. Throughout the paper, we maintain the distinction between shadow performance (paper-trading against historical data) and live performance (actual execution). The system currently operates on a demo account with real-time market data. No results presented here imply fitness for real capital.

---

2. Background: From Snake to Organism Vivo 2.1 The Snake Era (2018-2020) The project's earliest incarnation was a rule-based bot called "Snake," named for the serpentine motion of price within regression channels on 1-minute charts. Snake used three regression channels of different windows (240, 60, and 15 candles) to measure trend, tone, and entry timing. The system achieved high win rates (reportedly 81% in favorable conditions) but suffered from a critical weakness: it failed when market regimes changed. The failure mode was instructive. A bot that performs well "for a couple of months" is not necessarily a bad bot—it is a blind strategy. Without a shadow running in parallel, without temporal discipline in validation, and without the restraint to avoid retraining with the very regime that is causing losses, no amount of code quality can save the system. "1,756 lines of code functioning for a couple of months is not bad code—it is a strategy without eyes." This lesson became the foundation for Organismo Vivo's design principles: every decision must be tested against unseen future data; every claim of edge must be validated through walk-forward protocols; and the model that goes into production must remain frozen until evidence accumulates that the regime has fundamentally changed. 2.2 Design Philosophy Three principles guided the architecture: Principle 1: Learn, don't draw. The system avoids hand-coded indicators (moving averages, support/resistance, order blocks) as direct

signals. Instead, structural features must emerge from representations learned on raw price data. Traditional concepts are used only as baselines for comparison, never as components of the trading logic. Principle 2: Survive, don't predict. The primary goal is not maximum profit but minimum destruction of capital. The system is designed to skip signals when confidence is low, even at the cost of missed gains. The relevant metrics are not net R alone, but drawdown avoided and risk-adjusted efficiency. Principle 3: Adapt, don't retrain. When market regimes change, the model's learned representations may become obsolete. The temptation is to retrain on recent data. Our walk-forward experiments showed the opposite: retraining with recent data degrades performance because the new regime contaminates the learned representations. The operational consequence is that production models remain frozen; degradation is detected through monitoring, and adaptation occurs through threshold adjustment rather than weight updates.

Section 3: Methodology (EXPANDED) 3.1 System Architecture Organismo Vivo comprises four cooperating modules: OV (Organismo Vivo core): A reinforcement learning agent (PPO) that selects trade sizes based on "respiratory states" of the price. Magic number 777001. Currently operating on demo account. RC (Regression Channel): A port of a classical MetaTrader 5 Expert Advisor (EA) to Python, gated by a PPO agent that decides whether to skip signals or enter with risk levels of 0.5, 1.0, or 1.5. Magic number 25122013. AE (Agente Estructura): An encoder-based regime detector using contrastive learning (InfoNCE) on raw price windows, with a k-nearest-neighbor scoring mechanism in the latent space. Magic number 777003. Hermes Supervisor: A meta-agent with persistent memory (across sessions) that monitors the other modules, validates edge degradation, and provides human-readable reports via Telegram. Hermes itself is not an AI developed for this project—it is an off-the-shelf agent framework that we integrated as the supervisory layer. The modules communicate through shared CSV files and a centralized state machine. A watchdog process monitors all live components and restarts any that fail (39 restarts recorded over the project's lifetime, with high reliability). 3.2 The Respiratory State Framework The central representational concept of the project is that price exhibits measurable, repeating states analogous to biological respiration. We formalized this intuition into eight discrete respiratory states: Hyperactivity: Expanded amplitude, rapid rhythm Latency: Quiescence, accumulation Normality: Standard flow Apnea vera: Genuine stillness Apnea tensa: Tense stillness (pre-spasm) Spasm: Erratic movement without direction Fibrillation: High-frequency low-amplitude oscillation Deep respiration: Sustained directional movement with deep pullbacks The market is described as an organism with states, not as a price series. A macro-direction context (bull / bear / sideways) is layered on top to provide directional bias. States are computed from raw OHLCV data through a pipeline that includes volatility normalization, pivot detection, and structural classification. The exact feature engineering pipeline is documented in the project's private repository; for this paper, we focus on the validation methodology and results. 3.3 Encoder for Regime Detection (EXPANDED) 3.3.1 Architecture The AE module uses a 1D causal convolutional encoder. The architecture consists of three causal convolutional blocks followed by a linear projection head: Input: (batch, 64, 11) ← 64 M15 candles × 11 channels ↓ CausalConvBlock(in=11, out=32, kernel=5) + ReLU ↓ CausalConvBlock(in=32, out=64, kernel=5) + ReLU ↓ CausalConvBlock(in=64, out=128, kernel=5) + ReLU ↓ GlobalAveragePooling over time dimension ↓ Linear(128, 128) ↓ L2-normalize Output: (batch, 128) ← 128-dim embedding Total parameters: approximately 90,000. Causal padding: Each CausalConvBlock applies left-padding only (padding = kernel_size − 1 on the left side), ensuring that each output depends only on past and present inputs—never on future data. This property is essential for validity in any predictive task. 3.3.2 Input Representation The encoder receives a window of 64 M15 candles (16 hours of market data). Each candle is represented by 11 channels: 6 raw price channels: close-to-close return, normalized range, position within range, body relative to range, relative volume, relative spread. 5 structural channels (added in Phase 2): structural regime, distance to nearest swing, within-order-block flag, within-fair-

value-gap flag, liquidity sweep flag. All channels are clipped to [−5, +5] to handle outliers. 3.3.3 Contrastive Training Objective The encoder is trained with InfoNCE contrastive learning. Given a batch of N windows, the objective is to identify the correct positive pair for each anchor among N−1 negatives: $L = -1/N \times \Sigma_i \log(\exp(\text{sim}(z_i, z_{i+})/\tau) / \Sigma_j \exp(\text{sim}(z_i, z_j)/\tau))$ Where: z_i is the embedding of window i z_{i+} is the embedding of the positive pair (same market context, different augmentation) $\text{sim}(u,v) = u \bullet v / (\|u\|\|v\|)$ is cosine similarity $\tau = 0.07$ is the temperature parameter The temperature τ controls the concentration of the embedding distribution: lower values produce sharper distinctions between similar and dissimilar pairs. 3.3.4 Reaction Score via k-Nearest Neighbors For each closed candle, we compute a "reaction score" as the mean forward return over the next H candles of the K most similar historical candles in the latent space: $\text{score(candle)} = \text{mean}(r_{t+1} \ldots r_{t+h})$ for each of the K nearest neighbors Where: r_{t+i} is the close-to-close return from candle t+i−1 to candle t+i H = 16 candles (4 hours ahead) K = 50 nearest neighbors Distance metric: cosine distance in the 128-dim embedding space Temporal guard: Only neighbors whose timestamps are at least G candles in the past are considered, where G = 32 candles (8 hours). This guard ensures that the forward return window of the neighbor (16 candles ahead) and the decision point of the query (current candle) do not overlap. The relationship G > H is mathematically necessary and sufficient to prevent any look-ahead bias. 3.3.5 State Classification The reaction score, combined with two derived features, classifies each candle into one of four health states: vol_rel = ATR(14) / mean(ATR, 168) − 1 (volatility relative to weekly baseline) novelty = mean cosine distance to the 20 nearest neighbors (how unusual is the current context) The classification thresholds were determined empirically through backtest calibration, not through hand-tuning. 3.4 PPO Agent Architecture (NEW) 3.4.1 Action Space The PPO agent selects from a discrete action space: 0: Skip (do not enter the trade) 1: Enter with risk 0.5R 2: Enter with risk 1.0R 3: Enter with risk 1.5R The action space is intentionally limited to three non-zero risk levels to avoid overfitting to specific risk multipliers. 3.4.2 Observation Space The agent receives an observation vector that includes: 30 engineered features from the current market context (prices, volatility, structure) 3 position-related features (current PnL in R, holding time, distance to SL/TP) Total: 33 dimensions All features are normalized before being passed to the policy network. 3.4.3 Reward Function The reward is defined as: $R(\text{trade}) = \text{pnl\_R} \times \text{risk}$ $R(\text{skip}) = 0$ Where: pnl_R is the trade outcome in R-multiples (gain relative to stop distance) risk is the action's risk level (0.5, 1.0, or 1.5) Skipped trades receive zero reward The reward is only realized at trade closure, not at intermediate timesteps. 3.4.4 Training Protocol PPO training uses the Stable-Baselines3 implementation with the following hyperparameters: Hyperparameter Value Learning rate 3×10^{-4} Batch size 64 Number of epochs 10 Discount factor (γ) 0.99 GAE parameter (λ) 0.95 Clip range 0.2 Entropy coefficient 0.01 Total timesteps 500,000 per window Walk-forward validation used 12 overlapping windows of 500K timesteps each, with each window incorporating additional recent data. 3.4.5 Key Finding: Retraining Degrades The semiannual walk-forward experiment revealed that models retrained on data through 2025 consistently underperformed the frozen model (trained on data before 2025-01). Analysis of the divergence suggested that the 2025 regime "contaminated" the retrained models' representations of stable patterns. Operational consequence: Production models remain frozen. Edge degradation is detected through monitoring, and adaptation occurs through threshold adjustment rather than weight updates. 3.5 Multi-Skill Ensemble: Hermes (NEW) 3.5.1 The "Arms" Framework Hermes operates on a set of base signals we call "arms" (in the sense of weapons). Each arm is a scalar signal computed from the current market state: score: The kNN reaction score from the encoder (Section 3.3.4) reg: Structural regime indicator vol: Volatility regime (high/normal/low) spread: Current spread relative to ATR pos_rng: Position within recent range ret_fut: Forward return (used only in training, not at inference) 3.5.2 Skill Definition A "skill" is a rule that combines arms and generates a directional signal: signal = -1 if score < threshold_low signal = +1 if score > threshold_high
signal = 0 otherwise

(threshold_low, threshold_high) = percentiles of historical score distribution Skills can include filters (e.g., "only fire when vol > median") and combinations of arms. The search space is large but structured. 3.5.3 Skill Exploration Hermes explores the skill space systematically. For each candidate skill, it: Calibrates thresholds on the first half of the calibration period Evaluates out-of-sample on the second half (validation period) Reports the OOS performance without re-optimizing This protocol prevents the look-ahead bias that would result from threshold optimization on the evaluation set. 3.5.4 Ensemble Voting The final Hermes signal is produced by ensembling multiple skills. Each skill votes long (+1), short (−1), or flat (0). The ensemble fires a trade only when the sum of votes exceeds a threshold K: ensemble_signal = $\text{sign}(\Sigma_i \text{ skill}_i(\text{candle}))$ if $|\Sigma_i \text{ skill}_i(\text{candle})| \geq K$ else 0 The threshold K is calibrated to optimize PF on the calibration set, with a hold-out validation check to prevent overfitting. 3.5.5 Walk-Forward Validation Hermes skills are validated through walk-forward testing: Calibration window: First 50% of the validation period Evaluation window: Second 50% (unseen during calibration) Threshold recalibration: Only on the calibration half Performance reporting: Only on the evaluation half This protocol ensures that reported skill performance reflects generalization, not curve-fitting. 3.5.6 Statistical Validation For the best-performing skills, we compute: Bootstrap confidence intervals (95%) on PF and Sharpe ratio Permutation tests: Shuffle trade outcomes and recompute PF to assess significance Skills with bootstrap CIs excluding 1.0 and permutation p-values < 0.05 are classified as "statistically validated." 3.6 Validation Strategy The project employs a strict temporal discipline to prevent data leakage: Training cutoff: All models are trained on data with timestamps before 2025-01-01. Testing window: Validation occurs on data from 2025-07-01 onward (a full year of unseen data). Walk-forward: Models are evaluated across multiple rolling windows (typically 3-6 month horizons) to assess stability across regimes. 3.6.1 Out-of-Sample Testing The encoder's predictive power was validated on a held-out set never seen during training. Spearman rank correlation between the kNN score and forward returns was computed across multiple symbols and compared against two baselines: Model Spearman ρ Direction AUC Encoder (causal, live-deployable) 0.286–0.301 0.628–0.645 Raw features (causal baseline) 0.031–0.132 0.507–0.563 Encoder (non-causal, upper bound) 0.500–0.527 0.728–0.746 The gap between the causal encoder (0.29) and the non-causal upper bound (0.52) quantifies the headroom that look-ahead would provide; the fact that the causal encoder substantially exceeds the raw-features baseline confirms that the learned representations capture genuine predictive structure, not artifacts of the training procedure. 3.6.2 Walk-Forward Validation We validated the agent across multiple temporal windows: Regime walk-forward: Four windows covering distinct market conditions (COVID, bear, lateral, bull). Threshold for "passing": PF > 1.3. Result: 3 of 4 windows passed. Semiannual walk-forward: Twelve windows of 500K training steps each. The agent did not pass the 75% threshold for "winning windows." However, deeper analysis revealed a critical insight: models retrained on recent data performed worse than the original frozen model (see Section 3.4.5). 3.6.3 Preregistration of Hypotheses Following practices from experimental psychology and pre-registered clinical trials, we adopted preregistration for all post-fix live validation: Hypotheses (e.g., "PF_live > 1.0 over ≥30 trades") are documented before the validation period begins. Analysis plans are specified in advance. Any deviation from the plan is recorded as a dated amendment. This practice eliminates researcher degrees of freedom and protects against post-hoc rationalization. 3.6.4 Multiple Comparison Corrections We acknowledge that certain exploratory analyses (particularly the state × score calibration grid) were conducted post-hoc. We apply Bonferroni and Benjamini-Hochberg FDR corrections when reporting these results, and we explicitly flag which findings survive correction and which do not.

Section 4: Adversarial Audit Process 4.1 The Need for External Review Complex trading systems accumulate technical debt. Bugs that survive testing in development often surface only in production, where they can cause silent capital destruction. We adopted an adversarial audit model: an external auditor (in our case, another AI assistant acting in an adversarial role) was asked to attempt to falsify each claimed result. The auditor's mandate was explicitly: identify hidden look-ahead, flawed statistical reasoning, inflated effect sizes,

and execution assumptions that may not hold in live markets. 4.2 Bugs Found Through Auditing Seven production bugs were identified through adversarial auditing. We present them in detail because they illustrate the types of failure modes that can survive standard development testing. 4.2.1 Clock Offset Bug (Critical) Symptom: Live trades entered 181–192 minutes after the signal candle, when the system expected entry within 60 seconds. Root cause: MetaTrader 5 timestamps are encoded in broker time (UTC+3 for our broker), but Python's time.time() returns true UTC. The shadow system mixed these conventions, applying a 3-hour offset that did not exist. Evidence: All 7 live entries for NAS100 on a specific date (21/08/2026) were executed 181–192 minutes after the signal candle, with adverse price movement of −1.75 to −64.47 points, while the shadow showed positive returns on the same candles. Fix: Use tick.time from the symbol as the authoritative clock. Deployed and verified post-fix. Impact: This bug explains the divergence between shadow performance (+26.8R combined) and live performance (−32.30 USD) during the affected period. 4.2.2 Invalid Stops (retcode 10016) Symptom: 464 occurrences of stop-loss modification failures in the trading log. The trailing stop attempted to move SL to break-even, but the broker rejected orders where the new SL was too close to current price. Fix: Validate stop side and minimum distance from price before sending the modification request. 4.2.3 Magic Number Filter Failure Symptom: The MetaTrader5 Python library (version 5.0.6090) does not correctly filter by magic number. positions_get(magic=X) returns all positions regardless of the specified magic. Risk: Without filtering, one bot could close another bot's positions. Fix: All position access code now filters manually by magic number rather than relying on the library's broken filter. 4.2.4 Process Freezing on Launch Symptom: When launching background processes with output redirection via PowerShell's Start-Process, the shell task froze waiting for the child process to finish or buffer to fill. Fix: Launch without redirection; accept that output goes to a log file rather than the console. 4.2.5 EURUSD Structurally Inoperable Symptom: The minimum stop-loss distance configured for EURUSD (10 pips) was smaller than the broker's average spread (~13.5 pips), meaning the stop was inside the spread and would never trigger correctly. Fix: Increase EURUSD minimum stop distance and add a maximum-spread filter. 4.2.6 Look-ahead in Shadow Simulation Symptom: The shadow system filled stop-losses at the theoretical price even when a price gap jumped through the stop, introducing a favorable fill assumption. Status: Identified but not yet quantified. Future work will measure the impact on reported shadow performance. 4.2.7 Encoder Version Incompatibility Symptom: When the encoder architecture changed (from 6 to 11 input channels), the pre-computed kNN index became incompatible with the new embedding space. Fix: Rebuild the index with the current encoder before restarting the shadow system. 4.3 Lessons from the Audit Process The adversarial audit revealed that even careful development produces bugs that survive standard testing. Several of these bugs (clock offset, magic filter) would have caused silent capital destruction if not caught. We recommend that any production trading system undergo periodic adversarial review by an external party—whether human or AI.

Section 5: Philosophical Decisions 5.1 Why Manual Filters Fail The most controversial decision in this project is the refusal to use hand-crafted indicators or structures (order blocks, fair value gaps, moving averages) as direct signals. Structures must emerge from learned representations of raw price data. Four reasons support this position: Manual filters encode designer bias. Every hand-drawn rule (an order block, a moving average, a threshold) is an untested hypothesis injected into the system without validation. We prefer that structures emerge from the data through controlled statistical procedures. Manual filters do not scale. There are infinite combinations of indicators and thresholds. Exploring them manually is data mining disguised as analysis. The combinatorial explosion ensures that any pattern found in-sample will fail out-of-sample. Manual filters overfit. It is easy to find a rule that worked in the past; it is difficult to find one that will work in the future. Our preference for validation through temporal splits and walk-forward protocols protects against this failure mode. Functional learning outperforms geometric learning. The encoder learns predictive zones (functional) without "seeing" order blocks or swings as recognizable categories (geometric). The Silhouette score on the

latent space is approximately zero across regime, symbol, and future-direction partitions (−0.0095, 0.0018, 0.0002 respectively), confirming that the representations are not geometrically interpretable. Yet the same representations are functionally predictive. The edge lies in the function, not the geometry. A pragmatic note: RC does use explicit pivots (swing highs/lows) as part of its signal generation, and AE added structural channels (swing proximity, order-block proximity, fair-value-gap proximity) in Phase 2. These geometric features are used as context that the encoder learns to combine, not as direct signals. The distinction is subtle but important: context informs the embedding; signals emerge from the embedding. 5.2 "Not Losing Is Also Winning" The desired behavior is conservative gating: skip dubious signals and concentrate risk on high-confidence ones, even at the cost of missed gains. The metrics that matter are not net R alone, but losses avoided, efficiency per unit of risk, and drawdown avoided. In practice, the gating agent operates as follows: for each signal from the base strategy, the agent decides to skip (0), enter with low risk (0.5), medium risk (1.0), or high risk (1.5). The distribution of decisions is learned, not fixed. The opportunity cost of skipped signals is real—some skipped signals would have been winners. But the expected value calculation favors skipping when confidence is low, even at the cost of asymmetry in the win/loss distribution of skipped signals. 5.3 Retraining Degrades The most counter-intuitive finding of the project: retraining the model with recent data degrades performance. This emerged from the semiannual walk-forward experiment. Twelve windows, 500K training steps each. The frozen model (trained on data before 2025-01) consistently outperformed models retrained on data through 2025. Analysis revealed that the 2025 regime "contaminated" the retrained models: patterns that were stable in 2018-2024 became unstable when the model was forced to fit recent data, and the model's representations of stable patterns degraded. The operational consequence is significant: production models remain frozen. Edge degradation is detected through monitoring (rolling window comparison against baseline), and adaptation occurs through threshold adjustment (the Hermes supervisor) rather than weight updates. We acknowledge this finding runs counter to standard machine learning practice (where continuous learning is assumed beneficial). We believe the explanation lies in the non-stationarity of financial markets: the distribution shift that prompts retraining is often the same distribution shift that degrades the retrained model's performance on subsequent data. 5.4 The Role of AI Assistants This project was developed through close collaboration between a human trader and multiple AI assistants. The human provided vision, direction, philosophical grounding, and final decision-making. The AIs contributed experimental design, code, statistical analysis, documentation, and adversarial review. We note several limitations of this collaboration: Session-bounded memory: AI assistants do not retain information across conversation sessions unless explicit memory tools are used. Project context (paths, parameters, decisions) must be rediscovered each session, which introduces friction and risk of inconsistency. No persistent learning: AI assistants do not learn from the trades generated by the system. Each session begins with the same knowledge base, regardless of what the system has discovered. Non-verifiable reasoning: When an AI proposes a strategy or interpretation, it cannot prove its correctness outside of real execution. Confidence is an estimate, not a guarantee. Non-continuous execution: AI assistants cannot maintain a 24/7 process natively. The live trading agent operates independently of AI sessions; AI involvement is limited to design, supervision, and periodic review. These limitations do not invalidate the collaboration, but they define the appropriate role of AI: architect and debugger, not operator or production system. The agent can run autonomously once launched, but its design, supervision, and adaptation depend on human-AI interaction.

Section 6: Results 6.1 Honest Reporting Framework We adopt the following conventions for reporting results: Shadow performance refers to paper-trading against historical or live data, with explicit modeling of spreads and transaction costs. Live performance refers to actual execution on a demo account with real-time market data. All performance figures include estimated transaction costs (spread + slippage assumption). Edge magnitudes are reported qualitatively (e.g., "robust," "marginal," "degraded") when exact figures might enable

reproduction by competitors. 6.2 Encoder Predictive Validity The encoder demonstrates genuine predictive validity. On out-of-sample data never seen during training, the kNN reaction score achieves Spearman ρ of 0.286–0.301 across four symbols, substantially exceeding the raw-features baseline (0.031–0.132) while falling short of the non-causal upper bound (0.500–0.527). The direction-prediction AUC (0.628–0.645) confirms that the score ranks future price movements better than chance. This is not a large effect. A Spearman ρ of 0.29 means that the score explains roughly 8% of the variance in forward returns ($\rho^2 \approx 0.08$). But it is a consistent effect, present across multiple symbols and stable across time. 6.3 Edge Stability Across Assets Edge magnitude varies substantially across assets. Two assets showed robust edge over the validation period; two showed marginal or degrading edge; one showed insufficient signal. The heterogeneity is expected: different assets exhibit different microstructures, and a single framework cannot be optimal for all. A notable observation: the gating agent (RC) consistently outperformed its base strategy (the EA without gating) across all assets, even when the base edge was marginal. The value of gating lies not in generating edge where none exists, but in preserving capital when confidence is low. 6.4 Walk-Forward Results The walk-forward validation showed mixed results: Regime walk-forward: 3 of 4 windows passed the PF > 1.3 threshold. Semiannual walk-forward: Fewer than 75% of windows were profitable. The critical insight from the latter: retrained models underperformed the frozen model. We interpret these results as a realistic assessment of edge robustness. Edge exists, but it is not omnipresent; it comes and goes. The system's task is not to predict when edge will exist, but to exploit it when monitoring detects its presence and to scale down when it does not. 6.5 Edge Degradation and Adaptation Edge degradation is inevitable. Markets change; strategies that worked yesterday may not work tomorrow. The system's response to degradation is detection and adaptation, not retraining. The detection mechanism compares rolling window performance (40 trades) against baseline metrics. If WR drops more than 10% or PF drops more than 30% relative to baseline, an alert is triggered. The adaptation mechanism searches for threshold combinations in the recent trade window that would have maximized PF; if a combination with PF > 1.2 is found, it replaces the current thresholds. This adaptation is conservative: it adjusts when to act, not what the model knows. The model's learned representations remain frozen; only the decision threshold moves.

Section 7: Lessons Learned 7.1 Documented Weaknesses We enumerate weaknesses explicitly, following the principle that transparency about limitations increases credibility. Backtested edge magnitudes are potentially inflated. The shadow NAS100 performance figure, in particular, relies on assumptions about intra-bar tick ordering validated only against one day of logs (39 trades). Small samples for sub-strategy analysis. The best sub-strategy had n=99; live validation had n=21 trades; some state classifications had n=22. State ranking instability. The ordering of states by PF reversed within a 4-day window, indicating that the state × edge relationship is not stable enough to drive hard filtering decisions. Demo fills ≠ real fills. All live validation was on a demo account with idealized liquidity. Post-fix latency cost not measured. The live system still enters approximately 6 minutes after the shadow system, due to symbol rotation; the cost of this delay has not been quantified. Post-hoc calibration analysis. The score calibration grid (state × score strength) was explored post-hoc; corrections for multiple comparisons partially account for this. Fair comparison requires mechanism isolation. The Hermes-vs-AE comparison reused the same encoder to isolate the learning mechanism; without this, the comparison would conflate encoder quality with skill design. Survivorship bias in symbol selection. The five symbols were selected manually; results may not generalize to other instruments. Historical multiple hypothesis testing. Several strategies were explored and discarded informally before the current framework; the discarded strategies are not documented in detail. No online learning. The agents operate with frozen models; there is no mechanism to update learned representations based on live experience. 7.2 The Primary Risk Is Psychological The most important finding of this project is not technical. It is psychological. The trading strategy operates with a win rate of approximately 35%. This means that in any given week, the majority of

trades will be losers. A streak of 5-10 consecutive losses is common and expected within the system's statistical distribution. Such streaks can destroy the discipline of even experienced traders, leading to premature intervention (turning off the system, changing parameters, abandoning the strategy) at exactly the moment when statistical reversion favors the system. The system's architecture includes several features designed to mitigate this risk: The adaptation mechanism is automatic and does not require human approval. The performance comparison is rolling and pre-registered, reducing the scope for discretionary interpretation. The meta-agent (Hermes) provides objective summaries that can counteract emotional narratives. But architecture cannot eliminate psychological risk. The ultimate responsibility remains with the human operator: to trust the system through the losing streaks that the system's own statistics predict. 7.3 Three Generations, One Idea The project's evolution traces three generations of the same core idea: Snake (2018-2020): Channels + inclination. High win rate, no adaptation. Failed when regime changed. Snake Brand Dollar Vacuum Cleaner (2020-2022): Channels + respiration. 81% win rate reported. Still rigid; failed after months. Organismo Vivo (2024-present): Respiration + structure + ecosystem + RL risk management. Adaptive thresholds, adversarial auditing, frozen models with rolling validation. The throughline never broke. Each generation learned from the failure of the previous. The lesson is not that rigid systems are bad code—the lesson is that they are strategies without eyes. Adding eyes (shadow validation, temporal discipline, adaptation) is the entire contribution.

Section 8: Future Work 8.1 Edge Discovery (EVO) The current system validates and adapts existing edges. A natural extension is the automated discovery of new edges. We are developing a framework (EVO) where an AI agent explores combinations of features (price action, volume, structure, regime) without presupposing any specific pattern, validates candidate edges through walk-forward protocols, and reports only those that survive statistical correction. 8.2 Multi-Agent Portfolio Allocation The current architecture operates agents independently across assets. A meta-agent that allocates capital dynamically based on relative edge strength could improve risk-adjusted returns by concentrating exposure on assets with active edge and reducing exposure when edge degrades. 8.3 Persistent Memory for AI Assistants The lack of persistent memory in AI assistants is a structural limitation for long-horizon projects. Future work includes building a knowledge management layer that maintains project context, decisions, and lessons across sessions, reducing the overhead of context reconstruction.

Section 9: Conclusion This paper documented Organismo Vivo, a multi-agent trading framework developed through human-AI collaboration. The central contributions are methodological (a validation framework combining walk-forward testing, preregistration, and multiple comparison corrections), empirical (honest reporting of edge with explicit acknowledgment of limitations), and philosophical (the case for learned representations over manual filters, the surprising finding that retraining degrades, and the observation that psychological discipline is the primary risk). We acknowledge the limitations of our approach: small validation samples, potentially inflated backtested performance, and the structural constraints of AI-assisted research. We do not claim that the system is ready for real capital deployment. The system operates on a demo account, and all results presented here are intended to inform further research, not to guide investment decisions. The framework is a work in progress. We continue to accumulate live validation data, refine the adaptation mechanisms, and explore new approaches to edge discovery. We release this documentation in the hope that other researchers—human or AI—may find the methodology, the philosophical positions, and the honest accounting of limitations useful for their own work.

Section 10: Acknowledgments This work was co-authored by Augusto (Tato), a human trader, and multiple AI assistants including Claude (MiniMax), OpenWork, and Hermes Agent. The human provided vision, direction,

philosophical grounding, and final decision-making. The AIs contributed experimental design, code implementation, statistical analysis, documentation, and adversarial review. All financial decisions remain the responsibility of the human author. The system operates on a demo account; no results presented here imply fitness for real capital. We thank the adversarial auditor (Claude, in adversarial mode) for identifying seven production bugs during the audit process. We thank the meta-agent (Hermes) for providing continuous operational support. And we thank the open-source community for the libraries that made this project possible.

References

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.

Chen, T., Kornblith, S., Kornblith, K., Shlens, J., & Le, Q. V. (2020). A Simple Framework for Contrastive Learning of Visual Representations. Proceedings of the 37th International Conference on Machine Learning (ICML), PMLR 119, 1597–1607.

van den Oord, A., Li, Y., & Vinyals, O. (2018). Representation Learning with Contrastive Predictive Coding. arXiv preprint arXiv:1807.03748.

López de Prado, M. (2018). Advances in Financial Machine Learning. Wiley.

Bailey, D. H., & López de Prado, M. (2014). The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality. Journal of Portfolio Management, 40(5), 94–107.

Harvey, C. R., Liu, Y., & Zhu, H. (2016). ... and the Cross-Section of Expected Returns. Review of Financial Studies, 29(1), 5–68.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. arXiv preprint arXiv:1606.01540.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. Journal of Machine Learning Research, 22(268), 1–8.